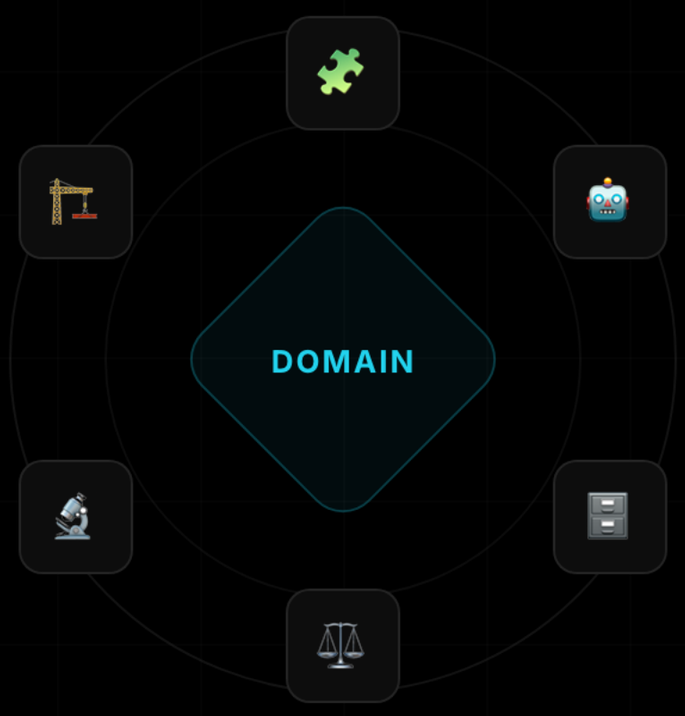
FREE DECISION FRAMEWORK



Hexagonal Architecture Decision Framework

The Gray Areas Guide

From the video: "Hexagonal Architecture Part 2, The Complete Implementation"

- When It's Overkill
- Framework Friction
- Mapping Strategies
- Conscious Shortcuts

Nil Monfort
AI Solutions Architect
youtube.com/@nilmonfort

PAGE 2

The Three Layers: Rules at a Glance + Testing Pyramid

⬡ DOMAIN

CENTER RING

- Entities & Value Objects
- Domain Services
- Port definitions (ABCs)

Rule: Zero external deps. Only Python stdlib + your own code.

"These are the things that make your app YOUR app, not just a wrapper around somebody else's API."

⬡ APPLICATION

MIDDLE RING

- Use Cases (one class per action)
- Orchestrates, never implements
- Deps injected via constructor

Pattern: constructor receives ports, execute() coordinates domain objects through ports.

"A use case should read like a recipe. If it reads like a novel, the domain model is too anemic." (Tom Hombergs)

⬡ INFRASTRUCTURE

OUTER RING

- Adapters (OpenAI, MongoDB, InMemory...)
- Composition Root (main.py)
- Mapping between domain & external formats

Rule: Implements port contracts. Never leaks external types into domain.

"Your domain entities serve the business. Your persistence entities serve the database. Don't make one class serve two masters."

DOMAIN EXAMPLE

```python
class Message(BaseModel):
    role: Role
    content: str
    timestamp: datetime

class LLMPort(ABC):
    @abstractmethod
    async def generate(
        self, msgs: list[Message]
    ) -> Message: ...
```

APPLICATION EXAMPLE

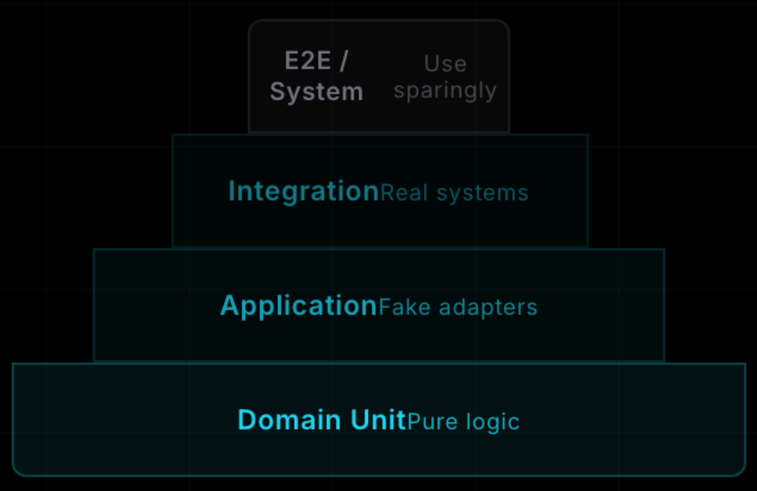
```python
class SendMessageUseCase:
    def __init__(self,
        llm: LLMPort,
        repo: ConversationRepo
    ):
        self._llm = llm
        self._repo = repo

    async def execute(self,
        conv_id, content
    ) -> Message: ...
```

INFRASTRUCTURE EXAMPLE

```python
class OpenAIAdapter(LLMPort):
    async def generate(
        self, msgs
    ) -> Message:
        # Map domain → OpenAI
        api_msgs = [
            {"role": m.role,
             "content": m.content}
            for m in msgs
        ]
        # Call API, map back
```

TESTING

The Pyramid That Builds Itself



Domain	Unit tests, **no mocks**, pure business logic. Test entities, value objects, domain services directly.	~ms
Application	Unit tests with **fake adapters** (not mocks). Verify use case orchestration flows correctly.	~ms
Infra	Integration tests with **real systems**. Test actual DB, API calls, file I/O against contracts.	~seconds
System	End-to-end tests. Full stack. Use sparingly: slow, brittle, expensive to maintain.	~minutes

★ **The test pyramid naturally emerges from the architecture. You don't enforce it.**

Real case: MADE.com swapped their entire persistence stack (Flask+SQLAlchemy+Postgres to CLI+CSV). Zero changes to domain or service layer.

PAGE 3

The Gray Areas Decision Matrix

1 When It's Overkill

- **Use hex arch:** Project > 2–3 days. Multiple integrations. Team > 1 person. Will swap providers.
- **Skip hex arch:** Utility script. Quick automation. Weekend prototype. Solo throwaway.

> *"With AI writing boilerplate, setup cost is basically zero."*

2 Framework Friction

Affected: LangChain, LangGraph, LangFuse, Phoenix

Problem: Ecosystem coupling makes clean hex arch hard. Workarounds needed to not break the core.

Decision: Comfort of the framework vs. clean everlasting code?

> *"You're batching their updates, relying on their release cycle. Your code is only as stable as their decisions."*

3 Where Do Prompts Live?

- **Infrastructure?** They're artifacts like training data, change frequently over time.
- **Domain?** They define behavior, they ARE business logic at runtime.

> *"Ask me where I put prompts in my last three projects and you'll get three different answers. No consensus yet."*

4 The Scaling Problem

- **Small project:** Hex arch works perfectly. Clean separation, obvious structure.
- **Large project:** Entities mix, ports folder becomes a mess, navigation suffers.
- **Solution:** Create **bounded contexts**. Separate domain/infra per use case group.

5 Mapping Strategy

STRATEGY	TRADEOFF	WHEN
No Mapping	Same model everywhere. Fast, but changes ripple across all layers.	Risky
Two-Way	Each boundary has own model. Changes isolated. Most teams land here.	Default
Full	Input/output models at every port. Max decoupling, max boilerplate.	APIs

> *"Mix strategies. Full mapping at API layer. Simpler mapping for stable internal boundaries."*

6 Conscious Shortcuts

- **MVP startup:** Skip web-to-domain mapping. Document it. Know when to reverse it.
- **Production medical system:** No shortcuts. Full separation, full mapping, full testing.

> *"The sin isn't taking a shortcut. It's taking one without understanding the cost. The enemy of good architecture isn't shortcuts. It's unconscious shortcuts."*

MYTHS

4 Common Misconceptions

✕ "Hex = DDD"

Reality: Orthogonal. Hex is architecture (layer boundaries). DDD is design methodology (ubiquitous language, aggregates). Use either without the other.

✕ "The hexagon IS the domain"

Reality: The hexagon contains BOTH application AND domain logic. Use cases live inside the hexagon too, not just entities.

✕ "It's overkill for everything"

Reality: Complex, long-lived systems with multiple integrations: it pays for itself. With AI writing boilerplate, setup cost approaches zero.

✕ "More mapping = waste"

Reality: Mapping cost is paid once. Independence and testability compound over years. The cost of NOT mapping ripples through every change.

WANT MORE LIKE THIS?

Build Software That Survives

A 3-part series on hexagonal architecture. From theory to production, file by file.

PART 1 ✓
Why Coupling Kills
Ports, adapters, and why the architecture exists.

PART 2 — YOU ARE HERE
Complete Implementation
Gray areas, testing pyramid, mapping strategies.

PART 3 → COMING
Real Production Project
Every file, every decision, free starter repo.

▶

Subscribe

Weekly deep dives on AI engineering, architecture, and production patterns.

youtube.com/@nilmonfort

🎬

Watch Part 3

Real production project walkthrough. Every file, every decision, explained.

Link in description

🚀

Starter Repo

Clone and start building with hexagonal architecture in under 5 minutes.

github.com/nilmonfort

Nil Monfort
AI Solutions Architect
youtube.com/@nilmonfort